Rappresentazione in base 2

NUMERI INTERI SENZA SEGNO

Conversione da base 2 a base 10:

somma delle potenze di 2 in base alla posizione che occupano

esempio 1:

$101011_2 = 1*2^0+1*2^1+0*2^2+1*2^3+0*2^4+1*2^5=1+2+0+8+0+32=43$

(dispari)

Video: https://www.youtube.com/watch?v=tFcmaxAtWJs

Conversione da base 10 a base 2:

divisioni successive per 2 e riscrittura dei resti…

esempio 2:

$50_{10}=110010_2$

		resti
50 : 2 =	25	0
25 : 2 =	12	1
12 : 2 =	6	0
6 : 2 =	3	0
3 : 2 =	1	1
1 : 2 =	0	1

Video: https://www.youtube.com/watch?v=qvfCkaQE_fA

NUMERI INTERI con SEGNO

(rappresentazione in Complemento a 2)

Gli interi con segno utilizzano la rappresentazione in C2:

il segno + è rappresentato con 0

il segno – è rappresentato con 1

Conversione di numeri **positivi** in C2:

- Convertire il numero in binario
- anteporre uno 0 alla stringa binaria
- NB: è meglio però riportarsi a formati fissi a 8, 16… bit

esempio 3:

Convertire il numero $+13_{10}$ in complemento a due (utilizzando 8 bit).

a) Converti 13_{10} in binario → 1101_2

b) Scrivi +13 in C2 anteponendo zeri → 00001101_{C2}

(il bit del segno è il primo a sinistra in rosso)

Conversione di numeri **negativi** in C2:

esempio 4:

Convertire il numero -15_{10} in complemento a due (utilizzando 8 bit).

a) Converti 15_{10} in binario → 1111_2

b) Scrivi +15 in C2 anteponendo zeri → 00001111_{C2}

(il bit del segno è il primo a sinistra in rosso)

b) Inverti i bit → 11110000

c) Somma 1_2 → 11110000 +

1 =

11110001_{C2}

+	0	1
0	0	1
1	1	10

Quindi -15_{10} è equivalente a 11110001_{C2} a 8 bit.

verifica (converto da C2 in base 10):

$1*2^0+0+0+0+1*2^4+1*2^5+1*2^6-1*2^7=-15$

(sottraggo la potenza di 2 di ordine maggiore, quella moltiplicata per il bit del segno)

esempio 5:

Convertire i numeri 0111_{C2} e 1111_{C2} (a quattro bit) in base 10.

Poiché il primo bit di 0111_{C2} è 0, si tratta di un numero positivo. Infatti:

$1*2^0+1*2^1+1*2^2-0=1+2+4=+7$

La conversione da C2 a base 10 per un numero positivo si esegue come se fosse un numero in base 2.

Il primo bit di 1111_{C2} è 1, quindi 1111_{C2} è un numero negativo; sommando le potenze di 2 e sottraendo quella di ordine maggiore (bit del segno) si ottiene:

$1*2^0+1*2^1+1*2^2-1*2^3=1+2+4-8=-1$

Numero di bit e rappresentazioni in binario e C2

Numero di bit n	Quanti numeri rappresentati: 2^n	Interi senza segno		Interi con segno (C2)	
		min	max 2^n-1	min -2^{n-1}	max $+2^{n-1}-1$
8 (1 byte)	256	0	255	-128	+127
16 (2 byte)	65.536	0	65.5365	-2^{15} -32.768	$+2^{15}-1$ +32.767
32 (4 byte)	4.294.967.296	0	$2^{32} - 1$ 4.294.967.295	-2^{31} -2.147.483.648	$+2^{31}-1$ +2.147.483.647
64 (8 byte)	2^{64}	0	$2^{64} - 1$ = circa $1,84 * 10^{19}$	-2^{63} -9.223.372.036.854.775.808	$+2^{63}-1$ +9.223.372.036.854.775.807

NUMERI DECIMALI (CON LA VIRGOLA)

SENZA SEGNO

Conversione da base 2 a base 10: si converte la parte intera e la parte decimale da base 2 a base 10

esempio 6:

$10111,011_2$

Conversione parte intera:

$10111_2=23_{10}$

Parte decimale: <u>somma di potenze di 2 decrescenti a partire dall'esponente -1</u>

$,011_2=0x2^{-1} + 1x2^{-2} + 1x2^{-3} = 0 + ¼ + 1/8 = (2+1)/8 = 3/8=0,375$

Quindi $10111,011_2$ è la rappresentazione in base 2 di 23,375

esempio 7:

$111,01_2$

Parte intera: 7

Parte decimale: $0x2^{-1}+1x2^{-2}=1/4=0.25$

Quindi 7,25

Conversione da base 10 a base 2: si converte la parte intera e la parte decimale da base 10 a base 2

esempio 8:

$35{,}625_{10}$

Conversione parte intera: $32+2+1=2^5+2^1+2^0=100011_2$

Conversione parte decimale: metodo delle moltiplicazioni successive (a destra della linea verticale si mette la parte intera del prodotto per 2, a sinistra la parte decimale); si leggono le "parti intere" dall'alto verso il basso

,625 x 2	1
,25	0
,5	1
$\boxed{,0}$	

$35{,}625_{10}$ è rappresentato in binario con $100011{,}101_2$

verifica: converto da base 2 a base 10

Conversione parte intera: $2^5+2^1+2^0=32+2+1=35$

Conversione parte decimale:

$1 * 2^{-1} + 0 * 2^{-2} + 1 * 2^{-3} = 1 * ½ + 0 * ¼ + 1 * 1/8 = ½ + 1/8 = (4+1)/8=5/8=0{,}625$

quindi la conversione è corretta.

esempio 9:

56,0625

Converto parte intera:

$56=32+16+8=111000_2$

Converto parte decimale (moltiplicazioni successive)

0,0625	0
,125	0
,25	0
,5	1
,0	

$56{,}0625_{10}$ in base 2 è $111000{,}0001_2$

verifica:

Parte intera: 32+16+8=56

Parte decimale:

0 x ½ + 0 x ¼ + 0 x 1/8 + 1 x 1/16=0,0625

RAPPRESENTAZIONE DEI NUMERI REALI IN FLOATING POINT

(rappresentazione in virgola mobile con standard IEEE 754)

Precisione *semplice* a 32 bit:

1	8	23
+/–	esp	mantissa

Precisione *doppia* a 64 bit:

1	11	52
+/–	esp	mantissa

Trattiamo per semplicità solo la rappresentazione in precisione semplice (float) a 32 bit.

Conversione da base 2 a base 10: si compila lo schema

$\pm 1.mantissa * 2^{esponente}$

convertendo

- il bit del segno
- l'esponente a cui si sottrae 127
- la mantissa

esempio 10:

Convertire in base 10 il numero

$11000000101000000000000000000000_{FP32}$

evidenziando il significato dei bit

1 10000001 01000000000000000000000

Il numero sarà ±1.*mantissa* * 2 *esponente*

1. Calcolo del segno:

primo bit 1 → segno negativo -

2. Calcolo dell'esponente:

$10000001_2=1*2^0+1*2^7=1+128=129$

129-127=2

3. Calcolo della mantissa:

$0{,}01000000000000000000000_2=2^{-2}=1/4=0.25$

Quindi il numero rappresentato è $-1{,}25*2^2=-5.00$

esempio 11:

$0\ 10000011\ 10011000000000000000000_{FP32}$

Il numero sarà ±1.*mantissa* * 2 *esponente*

1. Calcolo del segno:

primo bit 0 → segno negativo + quindi +1.*mantissa* * 2 *esponente*

2. Calcolo dell'esponente:

$10000011_2=1*2^0+1*2^1+1*2^7=1+2+128=131$

131-127=4

3. Calcolo della mantissa:

0, $10011000000000000000000_2=2^{-1}+2^{-4}+2^{-5}$ =
1/2+1/16+1/32+=(16+2+1)/32=19/32=0.59375

Quindi il numero rappresentato è +1, 59375*2^4=+25,5

esempio 12:

1 00110001 $01010100000000000000000_{FP32}$

Il numero sarà ±1.*mantissa* * 2 *esponente*

1. Calcolo del segno:

primo bit 1 → segno negativo -

2. Calcolo dell'esponente:

$00110001_2=1*2^0+1*2^4+1*2^5=1+16+32=49$

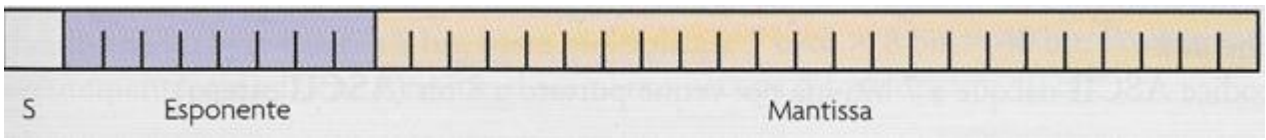
49-127=-78

3. Calcolo della mantissa:

0, $01010100000000000000000_2=2^{-2}+2^{-4}+2^{-6}$ =
1/4+1/16+1/64+=(16+4+1)/64=21/64=0. 328125

Quindi il numero rappresentato è -1,328125*2^{-78}

Conversione da base 10 in FP32: si compila lo schema



dopo aver convertito il numero reale in binario e dopo averlo NORMALIZZATO (parte intera 1,..... ovvero 1.*mantissa* * 2 ^{esponente}) si completano i 32 bit con:

1) 1 bit per il segno (come in C2)
   - 0 se + positivo
   - 1 se – negativo
2) 8 bit per l'esponente eccesso 127 (polarizzato o biased 127)
3) 23 bit per la mantissa

esempio 13:

Rappresenta in FP32 il numero +¾ ovvero +0.75

1) Conversione del segno + quindi il primo bit è 0

0																															

2) Rappresentazione della parte intera e parte decimale del numero:

0.75
Conversione parte intera: 0 quindi 0_2
Conversione parte decimale: $0.75_{10} = 11_2$

.75 x 2	1	↓
.5 x 2	1	
.0		

parte intera + parte decimale $= 0.11_2$

3) Normalizzazione: spostiamo la virgola verso destra in modo da lasciare un solo 1 a sinistra

mantissa

0.11 diventa $1.1 * 2^{-1}$

esponente da polarizzare

Esprimo l'esponente eccesso 127: -1+127=126

Convertendo 126 in binario:

0	**0**	1	1	1	1	1	1	0	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

esempio 14:

Rappresenta in FP32 il numero -11.375

1) Segno – quindi il bit del segno è 1

1																															

2) Conversione in binario del numero:

$11=1011_2$

0.375=0.011

.375 x 2	0	↓
.75	1	
.5	1	
.0		

1011.011

normalizzato diventa $1.\mathbf{011011_2}*2^3$

esponente +127=3+127=**130**

1	**1**	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**1**	**1**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

esempio 15:

Rappresenta in FP32 il numero -5.75

(0.75=0.5+0.25=.11)

Codifichiamo il numero -5.75 in singola precisione secondo lo standard IEEE 754.
Dobbiamo determinare segno, esponente e mantissa.
Poiché si tratta di un numero negativo il primo bit vale 1
Per determinare la mantissa scriviamo il numero in binario
101.11=1.0111 x 2^2. La mantissa è la parte a destra della virgola riempita con un opportuno numero di zeri. L'esponente vale 2, ma dobbiamo tenere conto del bias e quindi 2 → 129 che espresso in binario diventa 10000001

1 bit 8 bit 23 bit

S	Exp	Mantissa normalizzata
1	10000001	01110000000000000000000

esempio 16:

Converti in FP_{32} il numero +329.625

Segno:0

Conversione in binario:

$329_{10}=101001001_2$

0.625	1
0.25	0
0.5	1
0	

101001001.101 che normalizzato diventa

$1.01001001101*2^8$

8+127=135

$135_{10}=10000111_2$

Risposta:

0	10000111	0100100 11010000 00000000

esempio 17:
Converti in rappresentazione FP_{32}:

+13.75

esempio 18:
Converti +0.375 in FP32

esempio 19:
Converti da rappresentazione FP_{32} a base 10:

1 10000100 10010000000000000000000

$-1.mantissa*2^{esp}$

Esponente=132-127=5

Mantissa: 0.1001_2=1/2+1/16=9/16=0.5625

$-1.5625*2^5=-50$

esempio 20:
Converti da rappresentazione FP_{32} a base 10:

0	10000011	10011000000000000000000

esempio 21:
Converti da rappresentazione FP_{32} a base 10:

1	10000100	10010000000000000000000

esempio 22:

Rappresenta in FP32 il numero -49.526

1) Segno – quindi il bit del segno è 1

1																															

2) Conversione in binario del numero:

$49=32+16+1=2^5+2^4+2^0 \rightarrow 110001_2$

0.526

parte dec	parte intera	
.526 x 2	1	↓
.052 x 2	0	
.104 x 2	0	
.208 x 2	0	
.416 x 2	0	
.832 x 2	1	
.664	1	
.328	0	
.656	1	
.312	0	
.624	1	
.248	0	
.496	0	
.992	1	
.984	1	
.968	1	
.936	1	
.872	1	
.744	1	
.488	0	
.976	1	
.952	1	
.904	1	
.808	1	
.616	.....	

110001.10000110101001111101111 normalizzato diventa

1.10001100001101010011111~~101111~~.... $*2^5$

Polarizzazione dell'esponente=esponente+127=5+127=132

132=128+4=2^7+2^2=10000100

1	1	0	0	0	0	1	0	0	1	0	0	0	1	1	0	0	0	0	1	1	0	1	0	1	0	0	1	1	1	1	1

Quindi:

> Con un numero finito di cifre (in questo caso 32 bit) è possibile rappresentare solo un numero razionale che approssima *con un certo* ***errore*** il numero reale dato.

Convertitore da binario a decimale IEEE754

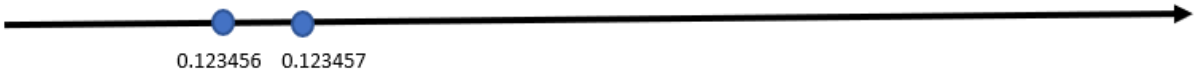
https://www.h-schmidt.net/FloatConverter/IEEE754.html

Convertitore da decimale a binario IEEE754

https://www.exploringbinary.com/floating-point-converter/

Esercizi

http://corsiadistanza.polito.it/corsi/pdf/09ASOP/pdf/base_n_fp.pdf



0,123456

Decimal

Double:

0.12345599999999999629718416827017790637910366058349609375

Single:

0.12345600128173828125

Raw Binary (sign field | exponent field | significand field)

Double:

0 01111111011 1111100110101100111111111010011111101011011010111111

Single:

0 01111011 11111001101011010000000

0,123457

Decimal

Double:

0.12345699999999999729727306885251891799271106719970703125

Single:

0.12345699965953826904296875

Raw Binary (sign field | exponent field | significand field)

Double:

0 01111111011 1111100110101110000011000001011101100101011101110101

Single:

0 01111011 11111001101011100000110

Per ottenere <u>un'approssimazione</u> migliore potremmo usare una rappresentazione a 64 bit (*double*) o a 128 bit.

Float, double, long double

L'esempio precedente si riferisce a un numero di tipo float o *singola precisione.*

Lo standard ANSI/IEEE754 definisce anche il numero di bit per ciascun campo per rappresentare numeri di tipo double (*doppia precisione*) e long double (*quadrupla precisione*).

Float

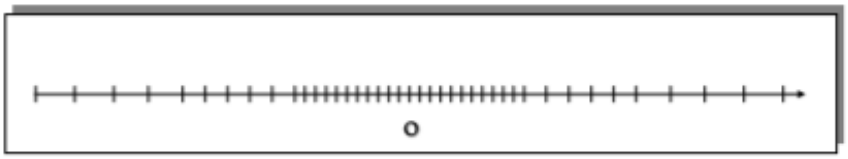
S (1 bit)	E (8 bit)	F (23 bit)

Double

S (1 bit)	E (11 bit)	F (52 bit)

Long double

S (1 bit)	E (15 bit)	F (112 bit)

Commenti

- Osservazione: in maniera assolutamente indipendente dalla rappresentazione usata, con 32 bit possiamo rappresentare "solamente" 2^{32} oggetti diversi
- I numeri rappresentati in virgola mobile hanno una densità maggiore vicino allo zero:
  - un numero che non sia rappresentabile viene approssimato al valore rappresentabile più vicino